

```
(r"/", MainHandler),
(r"/websocket", WebSocketHandler),
])

if __name__ == "__main__":
    application.listen(8888)
    tornado.ioloop.IOLoop.instance().start()
```

Tornado is still undergoing heavy development, but the developers say that the code has been cleaned up enough so that, theoretically, only the parts of Tornado that need to be used have to be imported. It's worked pretty well for me so far, so in the first three lines, I'm importing the core web module, the websocket module and the IO loop module (which ties all the modules together and runs the application).

Next up is the main handler class. In tornado, you have a bunch of classes, which define the methods you want to support as a function. In this case, the *MainHandler* class supports the HTTP GET method, and thus, I've sub-classed the *tornado.web.RequestHandler* class (which is the root class you need to sub-class in order to create a serve-able object) and overridden the *get()* method, so that it delivers the *front.html* page to the browser.

Then comes the WebSocket handler, which is the same deal as the *MainHandler*, except that WebSockets is a message-based protocol, so the only method you have to override is the *on_message()* function. The *on_message()* function is an event handler, which fires every time a message is received. As such, in our little program, all it does is echo back (using the *write_message()* method) whatever we've said, as a Unicode string.

Next up, I've instantiated the application, and mapped the / URL to the Main Handler, and the */websocket* path to the WebSocket handler. In the next if-clause (which, for Python novices, is the equivalent of C's *main()* function), I've instructed the app to listen on port 8888, and started the IO loop. Once you execute this file, our server is running.

Now, we've got to interface with this on the client side. Note that I'm using the standard interface, so the only browser this will work on now is Google Chrome/Chromium. It should work on Firefox 11 and above when it's released, or you can use a 'moz' prefix to the WebSocket object to make it work with versions of Firefox later than version 6 (use *MozWebSocket* instead of *WebSocket* in the JavaScript).

Without further ado, here's the *front.html* file, with the script embedded in it:

```
<!DOCTYPE html>
<html>
  <head>
    <title>WebSockets Demonstrator For LFY</title>
```

```
</head>
<body style="font-family: Lektion; font-size: 24pt">
  <p style="width: 800px">Powered by WebSockets. Use the
  form below to chat with the server.</p>
  <div id="chatbox" style="font-size: 14pt; height: 500px;
  width: 800px; overflow: scroll; border: 1px solid black"></div>
  <form id="conversation" onsubmit="DispatchText()"
  action="javascript:void(0);">
    <input type="text" id="message" name="message"
  autocomplete="off" style="width:780px" />
    <input type="submit" id="sub" name="sub"
  value="Send" style="width:90px" />
  </form>
  <script type="text/javascript">
    var ws = new WebSocket("ws://localhost:8888/
  websocket");
    ws.onmessage = function(evt){
      x = document.createElement("p");
      x.innerHTML = evt.data;
      document.getElementById("chatbox").
  appendChild(x);
    }

    function DispatchText(){
      var userInput = document.
  getElementById("message").value;
      document.getElementById("message").value = "";
      x = document.createElement("p");
      x.innerHTML = "You sent: " + userInput;
      document.getElementById("chatbox").
  appendChild(x);
      ws.send(userInput);
    }
  </script>
</body>
</html>
```

Before I start explaining this piece of code, let me make it very clear that to me, JavaScript is Satan himself. As a result, the code you've seen above has the potential to be very ugly. With that out of the way, let me explain what I've done.

The file is standard HTML5 with a valid Doctype (you need to use HTML5 specifically, because WebSockets is an HTML5 feature). There're all sorts of in-line styling, because that's one less file to send off to LFY to print. Believe me, the number of mistakes in printing is directly proportional to the number of files my article is split into. The message is: in production code, keep your styling information separate. Use a CSS file. I've also used the Lektion font and large sizes—this keeps the screenshot legible. Lektion isn't a Web-safe font. I have it installed, but most don't. However, it's available from Google Web Fonts if you want to use it, so it's all good!